Partial Reproduction of LiTS Math Search

A controlled modification of answer extraction rules

FTEC5660 Agentic AI for Business and FinTech

GitHub account fancecy
Revised 8 November 2026

1 Project summary and reproduction scope

The objective is a partial reproduction of the LiTS math search workflow, followed by a controlled modification of answer extraction rules. The local setup uses a pinned Qwen2.5 1.5B model. It completed ten fixed numerical MATH500 questions at three random seeds, giving 30 task executions. A controlled answer-extraction modification then re-scored exactly the same saved outputs. Mean extracted-answer accuracy changed from 16.7% (SD 15.3 percentage points) to 30.0% (SD 10.0 percentage points). This result demonstrates sensitivity to accepted answer formats, with no change to the generated reasoning. The paper's benchmark result is a reference rather than a directly replicated score.

LiTS separates policy generation, state transitions and reward scoring so that search algorithms can reuse task components [1, 2]. In the selected math workflow, a policy proposes reasoning steps, a transition updates the current state and checks termination, and a generative reward model scores candidate steps. Monte Carlo tree search (MCTS) selects and explores branches. These model-generated scores are guidance, not guaranteed mathematical correctness.

I selected the original runnable implementation so that configurations, search trees, terminal text and model usage could be inspected. The reported capability being tested is the authors' documented MCTS math search workflow on MATH500. I partially reproduce that workflow under laptop constraints, then measure the isolated effect of changing answer extraction on saved outputs. The extraction failure is an observation from this local study, not a result claimed by the original paper. The project does not reimplement MCTS or train a new model.

The paper's section 3.3, Table 6 reports 37% ReST-MCTS accuracy on 100 numerical MATH500 examples using Llama3-8B-Instruct [1]. My smaller model, ten-question subset, reduced search budget and local evaluation adapter differ. Sections 3 and 4 define the reproduction boundary and present the local baseline alongside that reported number; section 5 isolates the modification.

2 Setup notes

Item	Recorded setup
Upstream commit	25e94d862cb4514c22d9a80a635e392461769df7
Local model	Qwen/Qwen2.5-1.5B-Instruct
Provider and endpoint	Local Hugging Face checkpoint; no API base URL or key
Model revision	989aa7980e4cf806f80c7fef2b1adb7bc71aa306
Dataset	xinzhel/math500-float, pinned numeric subset
Dataset revision	d27dc61b1123c2fdc4eea855c802be87dfa9d536
Task subset	First 10 of 316 source-order numeric rows
Random seeds	42, 43 and 44, set at the run level
Hardware and execution	M4 MacBook Air, 16 GB host memory, CPU inference
Software	Python 3.13, Torch 2.15.0, Transformers 4.51.3
Search budget	2 MCTS iterations, 2 actions, depth 2, rollout length 1
Components	ConcatPolicy, ConcatTransition and GenerativePRM

The model and numerical dataset are pinned public releases [3, 4]. The selected execution environment could not access Metal acceleration. Search used four Torch threads and the model's automatic BF16 dtype. No paid model API was used. The software lock file records the observed dependency versions. Reference solutions were removed from the dataset loader. The concatenation transition starts from an empty state and does not expose the reference answer to generation; the scorer receives references only when checking correctness.

The nominal policy temperature is 0.7, top-p is 0.95, top-k is 50, and each policy generation allows at most 192 new tokens. At the forced depth boundary, upstream policy code switches to a near-zero temperature. Requested reward samples and termination samples are each one. Invalid reward formatting can trigger an additional next-token-logit scoring fallback. The 600-second check occurs before an iteration and cannot interrupt an individual long generation. These adaptations are shared by both scoring conditions. Both use the pinned repository prompts wrapped in the pinned Qwen chat template; re-scoring changes no generation prompt.

The dataset contains serialized Asymptote diagram code in two selected geometry questions. The text-only model received that code as part of the question rather than a rendered image. Three seeds repeat the same ten questions; they do not create thirty independent test questions.

3 Reproduction targets and metric definitions

Target	Success criterion
Reported workflow capability	Partially reproduce the documented MCTS math search pipeline
Local execution evidence	Complete 10 fixed questions at each seed and save configurations and trees
Controlled modification	Change extraction only and re-score identical saved outputs
Paper benchmark	Record 37% as context and disclose all comparison differences

The local baseline extracts answers with the authors' retrieve_answer_from_last_step helper and checks numerical equivalence with evaluate_math_qa. No terminal set or unparsed answer is removed from the planned denominator. Reference solutions are excluded from generation inputs; references enter only correctness checking.

Both conditions select a prediction by plurality among parsed terminal answers, with first-occurrence tie breaking. They then apply the same author-provided numeric correctness checker. The latest upstream CLI instead uses LLM voting through native tool calls, which the local Hugging Face wrapper does not support. This deterministic selection adapter is an explicit deviation, so 'baseline' below refers to the extraction-helper baseline within this adapter, not the paper's complete evaluation pipeline. Testing that full voting evaluator would require a separate compatible-model experiment.

Accuracy is correct extracted predictions divided by all ten planned questions in a run. Empty terminal sets, empty extractions and missing outputs stay in the denominator. I report the mean and sample standard deviation across the three run accuracies, plus the paired difference. A changed prediction can help or hurt. Token and time measurements describe shared generation; re-scoring adds zero generation calls.

For seed s, accuracy A_s equals the number of correct extracted predictions divided by 10. The reported mean is (A_42 + A_43 + A_44) / 3. Sample standard deviation and variance describe variation across these three run accuracies. The paired change is the modified accuracy minus the baseline accuracy for the same seed. A percentage-point change is a difference between percentages, not a relative percentage increase.

There are ten unique questions, repeatedly measured across seeds. The 30 executions are not thirty independent test questions, and no benchmark-wide significance claim is made. Empty terminal sets diagnose search or termination outcomes; unparsed answers diagnose extraction limits; wrong extracted values remain mathematical answer errors under the numeric checker. These categories help interpret the score but do not establish semantic correctness of every unparsed response.

4 Reproduction results against reported results

Measurement	Reported paper result	Local baseline
Extracted answer accuracy	37%	16.7% mean over three seeds
Model	Llama3-8B-Instruct	Qwen2.5-1.5B-Instruct
Questions	100 numerical examples	10 fixed source-order questions
Search iterations	10	2
Evaluation scope	Paper benchmark protocol	Extraction helper and local adapter

The table compares the reported benchmark reference with the observed local baseline. The two scores use different model, task and evaluation settings, so their numerical gap cannot be attributed to a single cause. The result supports successful execution of the adapted workflow; it does not establish a direct replication or a failure of the paper's 37% claim.

Seed	Baseline correct	Baseline accuracy
42	3 / 10	30.0%
43	0 / 10	0.0%
44	2 / 10	20.0%

All three planned runs completed without missing execution files. Baseline mean accuracy was 16.7% (SD 15.3 percentage points), with sample variance 0.023333 on the 0 to 1 accuracy scale. The baseline records five correct predictions over the 30 repeated executions. The following costs describe the shared generation used by both scoring conditions.

Shared generation cost	Measured total across all seeds
LLM calls	392
Input tokens	157,581
Output tokens	20,357
Reward logit fallback calls	47
Logged inference time	130.04 minutes
Launcher elapsed counter (perf_counter)	116.45 minutes
Extra generation calls for modified scoring	0

Logged inference time sums time.time-based model-call durations, including zero-output-token logit fallback passes. The launcher uses perf_counter. Their totals disagree: summed model durations exceed the launcher counter, so these clocks are not treated as interchangeable and no throughput claim is made. The reason for the discrepancy was not established. Formal generation runs execute sequentially. The original fields and logs are preserved for inspection.

5 Modification and results after modification

An exploratory terminal answered the divisor question correctly but concluded, 'So there are 9 positive whole-number divisors of 196.' The authors' numeric extraction helper expected a different answer marker and returned an empty string. This motivated an isolated evaluation modification. The initial search-budget comparison became diagnostic evidence; the formal study fixed generation at two iterations and compared extractors on identical outputs.

Rule	Baseline	Modified
Existing supported marker	Use author helper	Preserve helper result
Numeric boxed answer	No extra rule	Accept last numeric box
Explicit final conclusion	No extra rule	Accept an unambiguous number
Explicit count conclusion	No extra rule	Allow an object restatement
Ambiguous or unsupported text	Empty extraction	Empty extraction

The modified function receives no reference answer and makes no LLM calls. For an explicit count conclusion, a following 'of N' can restate the counted object, as in 'divisors of 196'; other numerical clauses remain unsupported. This is a conservative format extension, not a general semantic evaluator. The tests include the observed failure and negative examples with intermediate or conflicting numbers.

Seed	Baseline correct	Modified correct	Difference
42	3 / 10	4 / 10	+10.0 pp
43	0 / 10	2 / 10	+20.0 pp
44	2 / 10	3 / 10	+10.0 pp

Mean baseline accuracy was 16.7% (SD 15.3 percentage points). Mean modified accuracy was 30.0% (SD 10.0 percentage points). The paired difference was +13.3 percentage points (sample SD 5.8 points). Across the 30 repeated task executions, 4 changed from incorrect to correct and 0 changed from correct to incorrect under automatic scoring. These counts share ten unique questions. Across-seed sample variance on the 0 to 1 accuracy scale is 0.023333 for baseline and 0.010000 for modified scoring.

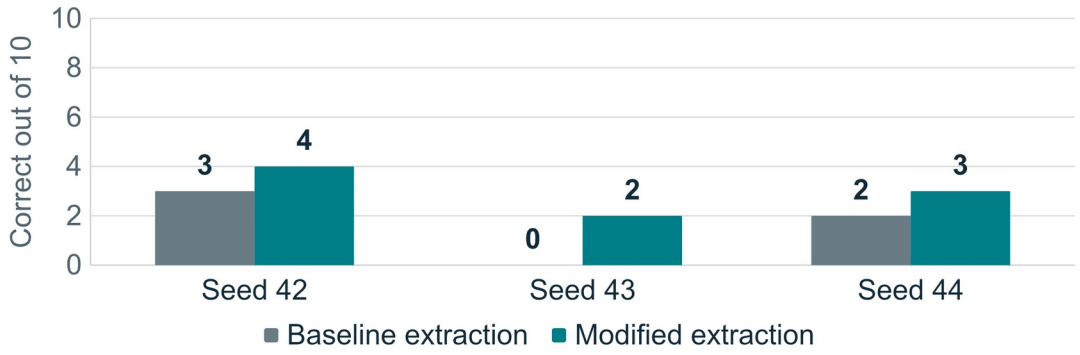


Figure 1 Correct extracted answers by seed out of ten fixed questions

Case audit and development overlap

Case	Observed behavior	Interpretation
Divisors of 196	Correct content 9 without the expected marker	Formatting can hide a valid answer
Regular hexagon at seed 42	Reference 42 with a supported answer marker	Both rules can agree on a correct answer
Cube from consecutive integers	A saved answer is 8 while the reference is 27	Extraction success does not imply reasoning success
Isosceles geometry	The new rule recovers 56 while the reference is 28	More parsed answers can still be wrong

The divisor conclusion is mathematically valid: both prime exponents are two, so the positive divisor count is nine. The hexagon has six sides of length seven, giving perimeter 42. For the cube question, the sum of three consecutive integers is divisible by three. Eight cannot satisfy this condition; 27 can. These examples separate a recovered correct answer, unchanged correct scoring and a genuine answer error. At seed 43, the hexagon answer instead ends with a natural-language conclusion giving 42 inches; the new rule recovers it while the baseline leaves it empty. At that seed, both cylinder terminals also explicitly give height 5 cm without the baseline marker; the new rule recovers the correct value from those final conclusions. In the geometry case, the model stops at the supplementary angle of 56 degrees and does not complete the isosceles-angle deduction. The modified rule faithfully extracts 56, but the numeric checker still marks it incorrect.

The audit file reviews all 6 changed question-level predictions, including the terminal text, applied rule and interpretation. Automatic correctness uses the fixed numeric reference. The audit separately checks whether the selected number actually serves as an explicit answer in the saved output. It does not silently change the primary scores or discard unfavorable cases. Reviewing changed predictions does not measure the semantic accuracy of unchanged unparsed outputs.

Question 0 also served as the development pilot. As a sensitivity analysis, excluding that question leaves nine unique questions and 27 task executions. Baseline scoring records 5 correct and modified scoring records 7 correct on those executions. This descriptive check makes development overlap visible. It is not an independently randomized or decontaminated held-out benchmark.

Output condition	Handling
No terminal answer	7 executions, counted incorrect
Policy calls at the 192-token limit	32 calls, a length-limit indicator
Missing execution files	None in the validated formal study

A terminal check can accept incomplete content, and some observed outputs end mid-sentence at the token limit. Length alone does not establish whether a generation ended naturally, so the capped-call count is a diagnostic proxy. The saved trees and raw text are needed to explain individual failures.

6 Debug diary

Blocker	Resolution and evidence
Python certificate validation	Used the system CA bundle without disabling TLS checks
CPU option rejected by MCTSConfig	Excluded the model-loading-only device field from search configuration
Quoted termination-list parsing	Used the repository-supported unquoted list syntax
Free text with step-signal termination	Corrected the incompatible pilot setting to binary sampling
Local model lacks native-tool voting	Disclosed a deterministic terminal-selection adapter
Valid answer lacks an expected marker	Tested an isolated extraction extension on saved outputs

The CPU compatibility fix applies to both evaluation conditions. Tests verify that CPU loading configuration works while genuinely unsupported search arguments still fail. The free-text termination pilots remain diagnostic records. My initial explanation that their empty terminals came only from search budget was incomplete; source inspection showed that the output format and terminal signal were incompatible.

7 Conclusions on reproducibility and limits

The workflow and re-scoring were reproducible in the tested environment: a pinned local execution of the authors' math-search workflow, together with saved configurations, trees, terminal outputs and re-scoring. The local exploratory run also revealed an extraction failure: a valid final answer can be absent from the numeric score when it lacks a recognized surface form. The extension changes that boundary without altering generation.

The paper's benchmark accuracy is not established here. A smaller model, short search budget, deterministic voting and a ten-question source-order subset limit comparison. Text-only treatment of diagram code is an additional input limitation. The rule was developed from one included pilot. I therefore interpret the results as a small reproducibility and evaluation study. A future study should test a larger independent sample and compare structured terminal outputs or the original voting interface as separate interventions.

For future users, I recommend pinning versions and question identifiers, validating termination and evaluation on a known example, retaining empty results in the denominator, and auditing raw answer content before interpreting a score change as a reasoning improvement. Saved outputs make these checks possible without paying for another generation run.

Reproduction links and references

The submission repository contains the original upstream code, the compatibility patch in a separate commit, the experiment scripts, completed evidence and presentation materials. Model weights and local caches are excluded. The README gives a clean installation and run path. No API key is required for the local experiment.

Submission repository

Experiment protocol and installation

Formal measurements and changed predictions

Answer extraction implementation

Reproduction sequence: create a Python 3.13 environment, install the pinned constraints, download the pinned public model with bootstrap_model.py, run test_local_study.py, execute run_study.py on a fresh study name, then run finalize_study.py and review the changed-prediction audit. CPU inference can take hours. The saved results permit evaluation review without repeating generation.

References

[1] Li, X., and Tao, Y. (2026). LiTS: A Modular Framework for LLM Tree Search. arXiv:2603.00631v2.

[2] LiTS source at the pinned upstream commit

[3] Qwen2.5 1.5B Instruct at the pinned model revision

[4] Numerical MATH500 dataset at the pinned revision

Tools included AI-assisted setup, coding and drafting. Saved executions and the case audit support the numerical claims. The presentation includes English notes and links to the same sources and evidence.